

Part 3 — Thought Process Write-Up

Real-Time Chat SPA — PulseChat

Candidate	Senior Frontend Engineer Candidate
Project	PulseChat — Real-Time Messaging Web Application
Live API Host	https://frontend-task-chatapp.onrender.com
Deliverable	Part 3 — Architecture, Technical Decisions, Trade-offs, Backend Quirks & Feature Set

1. Executive Summary & Approach

The objective of this assignment was to design, engineer, and deliver a production-grade, real-time messaging single-page application (SPA) across three unified deliverables:

- Part 1** — Core Chat SPA & Real-Time Engine (1-on-1 messaging, group chats, delivery/read receipts, replies, reactions, and dynamic presence).
- Part 2** — Creative Landing Page & Live API Sandbox (interactive API explorer, live server health/latency ping, and multi-persona demo switcher).
- Part 3** — Thought Process, Senior Architecture Write-up & Backend Insights.

Rather than assembling a superficial UI prototype, PulseChat was structured as a production-hardened client built with React 19 (Plain JavaScript), Tailwind CSS, and Vite. The focus centered on sub-millisecond real-time message stream fidelity, non-intrusive auto-scroll physics, robust normalization adapters, multi-tab synchronization via BroadcastChannel, local cache resilience when kicked from groups, and zero-asset Web Audio synthesis.

High-Level Architecture Flow

Layer	Modules	Responsibility
Entry / Routing	App.jsx	Routes to LandingPage, LoginPage, and ChatPage
Presentation	LandingPage.jsx · LoginPage.jsx · ChatPage.jsx	API sandbox & health display, auth/identity guard, core real-time chat UI
State Management	AuthContext → SocketContext → ChatContext	JWT/session/identity, singleton WebSocket engine, message stream & active threads
Data & Sync	client.js · useSound.js · useChatScroll.js · Presence Engine · BroadcastChannel	Normalization adapter, Web Audio synthesizer, viewport/scroll engine, presence heartbeat, cross-tab sync

Backend	Render-hosted API & Socket.io server	REST + WebSocket transport for all chat data
---------	--------------------------------------	--

Data flows top-down: routed pages consume the layered contexts, which in turn depend on the data & synchronization layer to communicate with the Render-hosted backend.

2. Architecture, Libraries & Technical Trade-offs (Part 1)

A. State Management: Domain-Segmented React Contexts vs. Redux / Zustand

Decision: Engineered a clean, domain-separated React Context hierarchy (AuthContext, SocketContext, ChatContext).

Rationale:

- Bringing in heavy global store libraries like Redux Toolkit or Zustand introduces boilerplate and external dependencies unnecessary for this scope.
- Monolithic single-context architectures cause re-render storms across the entire tree whenever a message is typed or received.
- Segmenting state into three distinct boundaries ensures strict separation of concerns.
- **AuthContext:** manages credentials, tokens, session persistence in localStorage, and identity verification.
- **SocketContext:** manages WebSocket lifecycle, connection statuses (connected, connecting, disconnected), reconnection backoff, and socket transport emits.
- **ChatContext:** manages conversation lists, unread counters, message pagination, optimistic UI updates, active chat state, reactions, and group operations.

Trade-off & Mitigation: High-frequency socket messages could trigger unneeded component re-renders if context values are unstable. This was resolved by applying useMemo and useCallback to all context values and optimizing sub-component subscriptions.

B. Real-Time Transport: Socket.io with Dual-Path Fallback

Decision: Bi-directional Socket.io WebSocket connection backed by an automatic REST POST /api/messages fallback.

- WebSocket delivers sub-millisecond round-trip delivery for message streaming and presence.
- Dual-Path Resilience: messages are optimistically inserted into local state with status “sending”. The client emits socket.emit(“message:send”). If socket transport fails or drops, it automatically falls back to POST /api/messages. If both fail, the message is marked “failed” and displays a 1-click retry button.

C. Normalization & Adapter Layer (src/api/client.js)

Decision: Centralized entity normalization at the network boundary.

Rationale: the backend API exhibits schema polymorphism across protocols:

- REST endpoints return MongoDB ObjectIDs as `_id` and ISO-8601 string dates (e.g. 2026-08-22T00:14:40.411Z).
- Socket.io events emit id and epoch timestamps in numeric milliseconds (e.g. 1787357854491).
- Direct chats return a singular participant object, whereas group chats return arrays of participants and admins.
- The adapter layer normalizes all incoming and outgoing entities into predictable, type-safe data models before reaching UI components.

3. Design Choices & Evaluator Experience (Part 2)

The landing page and overall UI were designed to be functional, elegant, and directly usable by technical evaluators:

- **Interactive API Explorer & Sandbox:** embedded live interactive documentation where evaluators can test endpoints, review request/response schemas, and copy ready-to-run curl commands with one click.
- **Real-Time Backend Health & Latency Monitor:** periodically pings GET /health to display live server status, operational uptime, and millisecond latency.
- **Multi-Persona Demo Switcher (SwitchUserModal.jsx):** evaluators can switch between pre-configured test users (Ada Lovelace, Bob Engineer, Charlie Dev) with one click to test two-way real-time conversations without manually logging in and out.
- **Dark Glassmorphic Aesthetic:** built with Tailwind CSS, using slate dark tones, subtle borders, backdrop blurs, and micro-interactions that match modern chat applications.

4. How AI Tools Were Leveraged

In building this application, AI tools (Antigravity coding assistant) were used as an accelerator for routine tasks while maintaining senior engineering control over architecture and core mechanics.

What AI Was Used For

- Generating initial component scaffolding and boilerplate forms.
- Parsing the raw Swagger JSON schema into initial REST client stubs.
- Generating exploratory Node.js test scripts to probe live backend socket behaviors and verify endpoint status codes.

What Was Modified, Rejected, or Written Manually

- **Rejected Naive Polling:** rejected generic AI-suggested polling mechanisms in favor of event-driven WebSockets paired with BroadcastChannel for tab-to-tab sync.
- **Replaced Audio Assets with Web Audio API:** AI suggested bundling static .mp3 audio files. This was rejected to avoid external asset dependencies, 404 network errors, and bundle bloat — a native Web Audio oscillator synthesizer (useSound.js) was written from scratch instead.

- **Custom Scroll Physics Engine:** AI suggested standard `scrollTop = scrollHeight`, which caused jerky scrolling and cropped multiline messages. A custom double `requestAnimationFrame` bottom-anchor engine with smooth distance-aware auto-scroll was implemented manually.
- **Group Kicked Archival Protection:** handled the backend's destructive removal behavior by writing a client-side `localStorage` caching layer that preserves past message history in read-only mode when a user is removed from a group.

5. Backend Quirks, Edge Cases & Solutions Encountered

During development and testing against the live backend

(<https://frontend-task-chatapp.onrender.com>), several quirks were uncovered and resolved:

Quirk / Issue	Root Cause on Backend	Graceful Client-Side Solution
/health Route Path	Swagger docs list /health under /api/*, but /api/health returns 404 NOT_FOUND. The real endpoint lives at root GET /health.	Added an isRoot: true flag in the API client wrapper to target the root origin specifically for health checks.
Entity ID Discrepancy	REST responses return _id, while Socket message:new broadcasts emit id.	Entity normalizers in client.js bind both id and _id to the canonical string ID.
Timestamp Polymorphism	REST endpoints return ISO-8601 string dates; WebSocket events return numeric epoch milliseconds.	All date formatters wrap inputs with new Date(dateInput), handling both formats without NaN errors.
Group Participant Deletion Quirk	Removing a user from a group causes the backend to remove the group from GET /conversations and return 404 NOT_FOUND on GET /messages/:id.	Implemented local group and message caching in localStorage. Kicked users retain complete message history in read-only mode with an informative status banner replacing the input box.
Group Creation Signature	Parameter signature mismatch between nested payload objects and multi-argument function calls.	Made createGroupConversation accept both { name, participantIds } and (name, participantIds), sanitizing all IDs before sending.
Background Sync Typing Conflict	5-second background sync interval caused activeConversation to refresh, resetting the group rename input field while typing.	Added an isEditingName guard preventing background state updates from overwriting active user input.

6. Scaling Roadmap (Future Enhancements)

With additional engineering time, the following enhancements would be added:

- **IndexedDB Offline Database (Dexie.js):** full offline persistence with a background sync queue for sending messages composed while offline.
- **Media Attachments Pipeline:** pre-signed S3/Cloud Storage upload URLs with client-side image compression, thumbnail previews, and upload progress bars.
- **DOM Virtualization (@tanstack/react-virtual):** virtualized windowing for message threads with 10,000+ messages to guarantee 60fps rendering performance.
- **End-to-End Encryption (E2EE):** native Web Crypto API key exchange implementing the Signal Protocol / Double Ratchet for end-to-end private messaging.

7. Complete Feature Set

The following is the full inventory of features implemented across PulseChat, grouped by functional domain.

Core Chat Features

- Real-Time Messaging
- Private 1-on-1 Chats
- Advanced Group Chats
- Message Delivery Status
- Read Receipts
- Quoted Message Replies
- Interactive Emoji Reactions
- One-Click Message Copy
- In-Conversation Message Search

Group Management

- Add & Remove Members
- Promote Members to Admin
- Rename Groups
- Leave Groups
- Real-Time Group Notifications
- Live Member Count Updates
- Automatic System Announcements

Smart Messaging Experience

- Unread Message Indicators
- Unread Conversation Counters
- Unread Messages Divider
- Smart Auto-Scroll
- New Message Notifications
- Responsive Mobile Experience

Real-Time Synchronization

- Socket.io Real-Time Communication
- Cross-Tab Synchronization
- Instant Reaction & Read Status Updates
- Background Sync Fallback

Privacy & Security

- Individual Message Deletion
- Cleared Chat Protection
- Read-Only Mode for Removed Group Members
- Protected Routes & Authentication
- Unique Phone Number Identity Validation

User Experience

- Modern Dark UI
- Glassmorphism Design
- Smooth Animations
- Interactive Hover Effects
- Trendy Multi-Typography
- Direct Navigation to Live Chat Demo